

포트폴리오

조성원

010-9742-4028

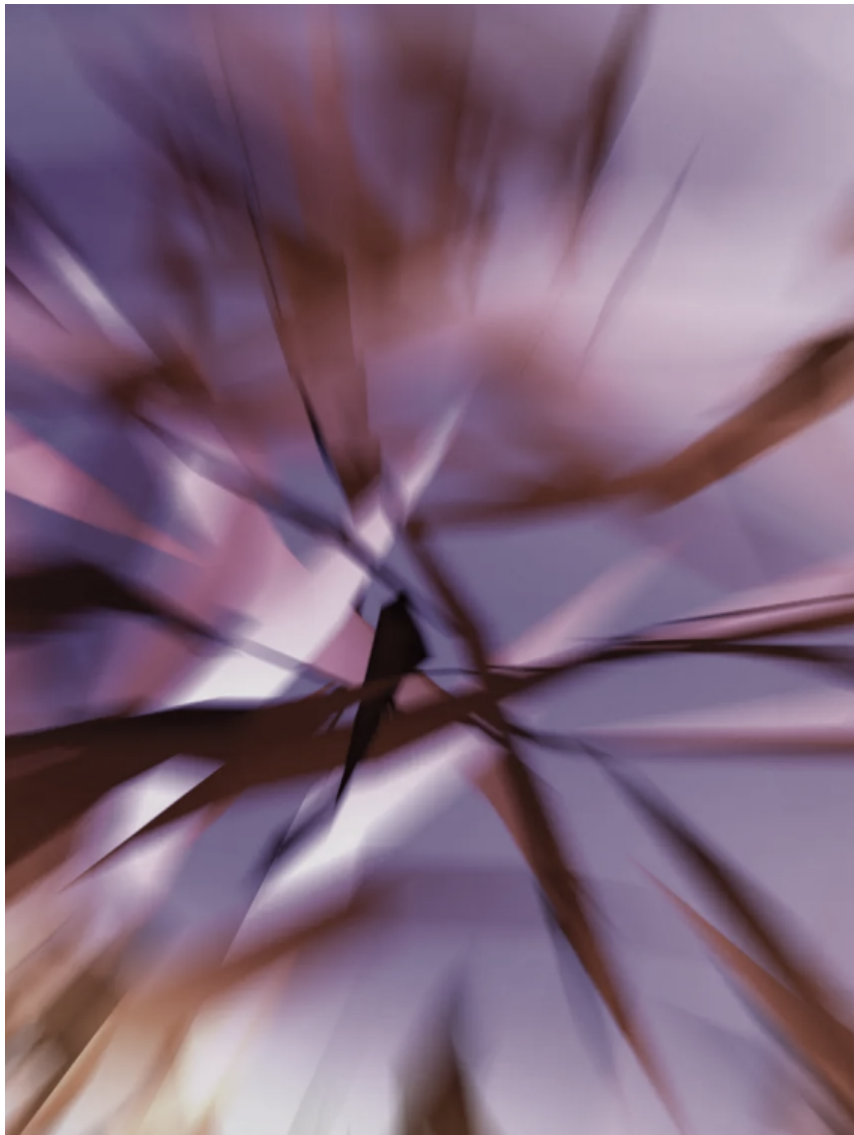
sungone000604@gmail.com

조성원

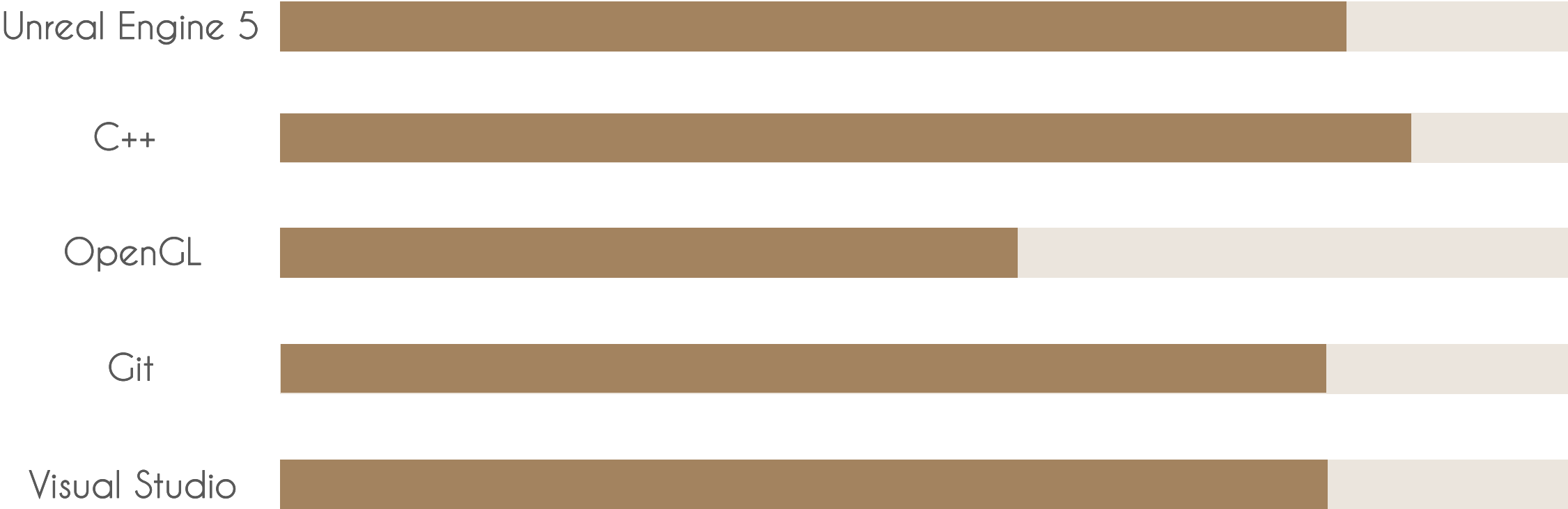
Programmer

+82 010-9742-4028

sungone000604@gmail.com



Tech Stack



CONTENTS

01. 티유월드	4p : 게임 소개 , 5p : 구현 설명
02. Unity Of Mind	12p : 게임 소개 , 13p : 구현 설명
03. 준비중	제목에 대한 간략한 설명을 쓰세요

Project 소개

티유월드

2024.09 ~ 2025.07

한국공학대학교 졸업작품

- 3D MMORPG
- Unreal Engine 5.4 개발
- 팀 3명 : 클라이언트 1, 서버 1, 모델러 1
- 클라이언트 파트 담당
- 학생의 실수로 소환된 몬스터들을 물리치고 학교의 평화를 되찾는 액션 RPG 게임

● 깃허브

<https://github.com/sungone/GP2025>

● 유튜브

<https://www.youtube.com/watch?v=NkqPZXgjn8c>



티유월드

플레이어 이동 동기화

내 클라이언트가 움직일 때 게임 상에 존재하는 모든 클라이언트들과 동기화 하기 위해 서버로 위치 패킷을 전달하는 **GPMYplayerNetworkSyncHandler** 모듈을 만들었습니다.

서버의 Overhead 를 고려하여
위치 업데이트가 꼭 필요한 상황에서만 패킷을 보내
도록 구현하였습니다.

1. Tick 마다 내 플레이어의 새로운 위치 정보를 갱신
2. 내 플레이어가 Idle 상태인지 확인해서 상태 업데이트
3. 점프, 제자리에서 회전 상황일 때 패킷 전송
4. 일정 주기마다 Idle 상태가 아닐 때 이동 패킷 전송

```
void UGPMYplayerNetworkSyncHandler::Tick(float DeltaTime)
{
    if (!Owner || !Owner->NetMgr) return;

    MovePacketSendTimer -= DeltaTime;

    Owner->CharacterInfo.SetLocation(Owner->GetActorLocation().X,
                                     Owner->GetActorLocation().Y, Owner->GetActorLocation().Z);

    Owner->CharacterInfo.Yaw = Owner->GetActorRotation().Yaw;

    const float CurrentSpeed = Owner->GetVelocity().Size();
    Owner->CharacterInfo.Stats.Speed = CurrentSpeed;

    HandleIdleState();
    Owner->UpdateGroundZLocation();
    HandleJumpState();
    HandleRotationChange();
    HandlePeriodicSend();
}
```

티유월드

플레이어 이동 동기화

서버에서 받은 다른 클라이언트들의 위치 정보는
0.5초 정도의 주기로 업데이트 되는데
내가 보는 월드에서 자연스럽게 업데이트 하기 위해
Location 값을 보간 처리하였습니다.

1. 최근에 받은 이동 패킷 TargetLocation 과 지금 내 위치 CurrentLocation , 이동 상태를 설정
2. 현재 위치에서 목표 위치까지의 거리 계산
3. 이동 및 회전 보간 처리
4. 이동 및 회전을 적용
5. 속도 계산 및 반영

전체 코드

<https://github.com/sungone/GP2025/blob/main/Source/GP2025/Character/GPCharacterBase.h>

```
void AGPCharacterBase::HandleRemoteMovementSync(float DeltaTime)
{
    if (DeltaTime <= 0.f)
        return;

    FVector CurrentLocation = GetActorLocation();
    FVector TargetLocation = CharacterInfo.Pos;

    const float SprintSpeed = 1200.f;
    const float WalkSpeed = 600.f;
    float Speed = CharacterInfo.HasState(STATE_RUN) ? SprintSpeed : WalkSpeed;

    FVector MoveDir = TargetLocation - CurrentLocation;
    float DistanceToTarget = MoveDir.Size();

    if (DistanceToTarget < KINDA_SMALL_NUMBER)
    {
        GetCharacterMovement()->Velocity = FVector::ZeroVector;
        return;
    }

    MoveDir.Normalize();

    float MoveDistance = FMath::Min(Speed * DeltaTime, DistanceToTarget);
    FVector NextLocation = CurrentLocation + MoveDir * MoveDistance;

    FRotator TargetRotation(0.f, CharacterInfo.Yaw, 0.f);
    FRotator InterpolatedRotation = FMath::RInterpTo(
        GetActorRotation(),
        TargetRotation,
        DeltaTime,
        10.f
    );

    SetActorLocationAndRotation(NextLocation, InterpolatedRotation, false);

    FVector NewLocation = GetActorLocation();
    FVector DeltaMove = (NewLocation - CurrentLocation);

    if (DeltaMove.SizeSquared() > KINDA_SMALL_NUMBER)
    {
        FVector NewVelocity = DeltaMove / DeltaTime;
        GetCharacterMovement()->Velocity = NewVelocity;
    }
    else
    {
        GetCharacterMovement()->Velocity = FVector::ZeroVector;
    }
}
```

티유월드

서버에서 수신받은 패킷을 게임 월드에 반영

서버에서 수신한 패킷 데이터를

UGPObjectManager 클래스를 통해 게임 오브젝트
에 반영하였습니다.

게임 월드 내의 모든 플레이어, 몬스터, 아이템은

UGPObjectManager 에서 고유 ID 와 객체 포인터
로 매핑되어 관리되며, 서버에서 전달된 상태 변화 및
이벤트(이동, 공격, 체력 감소,아이템 스폰, 아이템 구
매, 친구 등)를 타겟 오브젝트에 동기화 하기 위해 각
상황에 맞는 함수를 만들어 처리하였습니다.

```
void SetMyPlayer(AGPCharacterMyplayer* InMyPlayer);

void AddMyPlayer(const FInfoData& PlayerInfo);
void AddPlayer(const FInfoData& PlayerInfo);
void RemovePlayer(int32 PlayerID);
void UpdatePlayer(const FInfoData& PlayerInfo);
void DamagedPlayer(const FInfoData& PlayerInfo);
void HandlePlayerDeath(int32 playerId);

void PlayerAttack(int32 PlayerId, FVector PlayerPos, float PlayerYaw);
void PlayerUseSkillStart(int32 PlayerID, ESkillGroup SkillGID, float PlayerYaw, FVector PlayerPos);
void PlayerUseSkillEnd(int32 PlayerID);
void SkillUnlock(ESkillGroup SkillGID);
void SkillUpgrade(ESkillGroup SkillGID);
void LevelUp(const FInfoData& PlayerInfo);

void AddMonster(const FInfoData& MonsterInfo);
void RemoveMonster(int32 MonsterID);
void HandleMonsterDeath(int32 MonsterID);
void UpdateMonster(const FInfoData& MonsterInfo);
void DamagedMonster(const FInfoData& MonsterInfo, float Damage);
bool IsCritical(float Damage);
void PlayEarthQuakeEffect(const FVector& RockPos, bool bDebug);
void PlayFlameBreathEffect(const FVector& Origin, const FVector& Dir, float Range, float Angle, bool bDebug);

void ItemSpawn(uint32 ItemID, uint8 ItemType, FVector Pos);
void ItemPickUp(uint32 ItemID);
void ItemDespawn(uint32 ItemID);
void DropItem(uint32 ItemID, uint8 ItemType, FVector Pos);
void AddInventoryItem(uint32 ItemID, uint8 ItemType);
void UseInventoryItem(uint32 ItemID);
void EquipItem(int32 PlayerID, uint8 ItemType);
void UnequipItem(int32 PlayerID, uint8 ItemType);

void SoundWhenBuy(EResultCode Code);
void ChangeChannel(const FVector& RandomPos);
void ChangeZone(ZoneType OldZone, ZoneType NewZone, const FVector& RandomPos);
bool IsChangingZone() const { return bChangingZone; }
void SetChangeingZone(bool value) { bChangingZone = value; }
FRotator GetDefaultZoneRotation(ZoneType Zone);
UFUNCTION()
0개의 청사진 참조
void HandleLevelUnloaded();
UFUNCTION()
0개의 청사진 참조
void HandleLevelLoaded();
void RespawnMyPlayer(const FInfoData& info);
void ShowTutorialStartQuest();
void OnQuestStart(QuestType Quest);
void OnQuestReward(QuestType Quest, bool bSuccess, uint32 ExpReward, uint32 GoldReward);
void PlayWorldIntro();
void PlayTinoIntro();
void StopLoginSound();
UFUNCTION()
0개의 청사진 참조
void OnWorldIntroFinished();
UFUNCTION()
0개의 청사진 참조
void OnTinoIntroFinished();
// Friend
void AddRequestFriend(const FFriendInfo& FriendInfo);
void RemoveRequestFriend(uint32 DBId);
void AddFriend(const FFriendInfo& FriendInfo);
void RemoveFriend(uint32 DBId);
void ShowFriendMessage(int32 Result);
uint32 GetFriendDBId(FString Name) const;
```

[전체 코드](#)

<https://github.com/sungone/GP2025/blob/main/Source/GP2025/Network/GPObjectManager.h>

티유월드

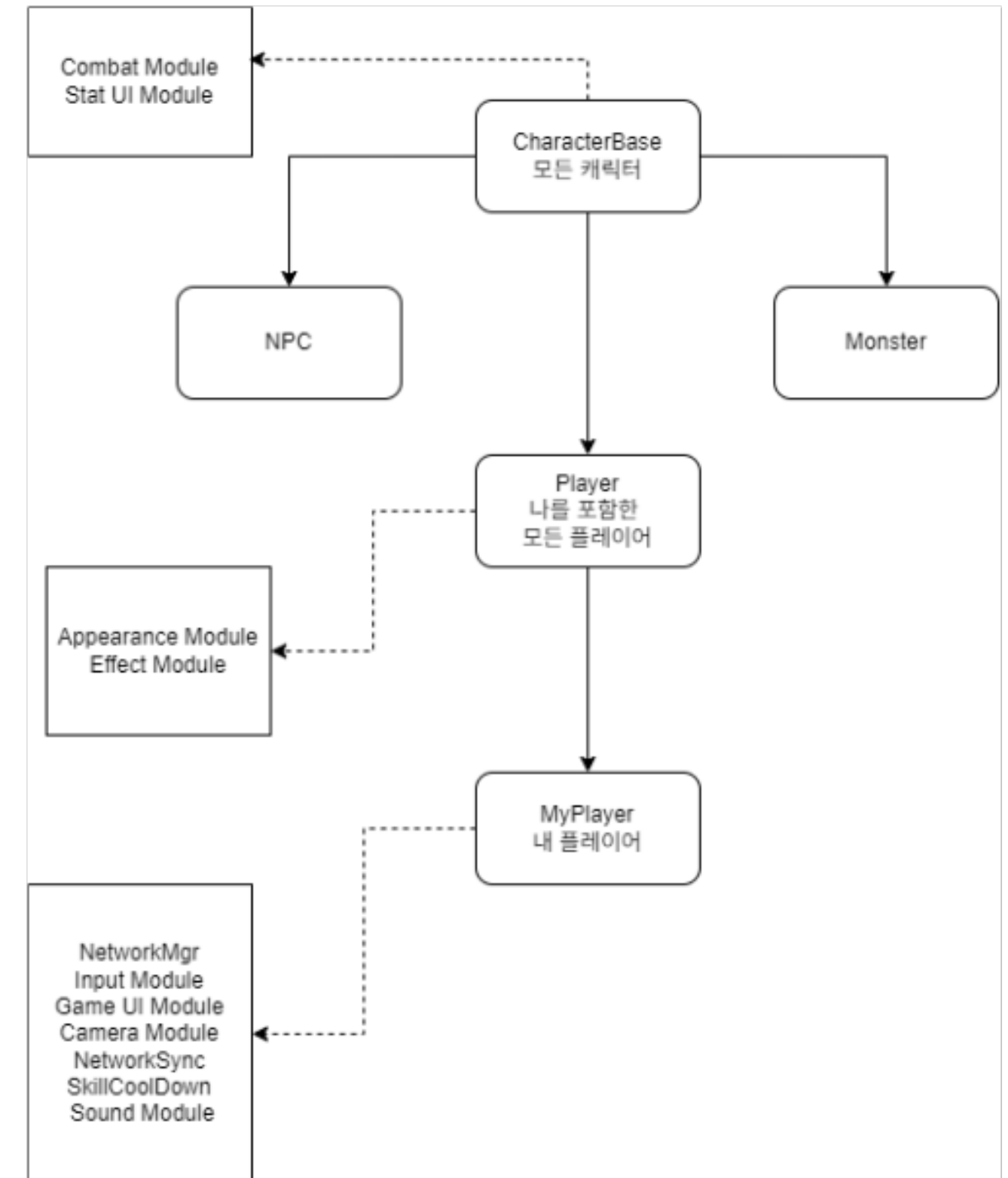
캐릭터 클래스 구조 설계 및 캐릭터 기능 모듈화

MMORPG 의 다양한 캐릭터가 공통적으로 가지는 기능은 AGPCharacterBase 에서 정의하고, 이를 상속받아 플레이어, 몬스터, NPC 로 세분화하였습니다. 로컬 플레이어의 입력 및 UI 처리를 위해 AGPCharacterMyplayer 클래스를 별도로 설계하였습니다.

캐릭터 클래스 내부에 모든 기능을 직접 구현하지 않고 전투, 스탯, UI 등 기능 단위를 모듈화하여 각 캐릭터 클래스가 필요한 모듈만 조합하도록 설계하였습니다. 이를 통해 유지보수성과 확장성을 높이고, 캐릭터별 특성을 잘 반영할 수 있도록 하였습니다.

전체 코드

<https://github.com/sungone/GP2025/tree/main/Source/GP2025/Character>



티유월드

캐릭터 전투 시스템 구현

UGPCharacterCombatHandler 모듈을 통해 MMORPG 캐릭터의 전투 시스템을 모듈화하여 구현했습니다.

이 모듈은 기본 공격, 스킬, 피격, 사망 처리와 관련된 애니메이션을 몽타주 단위로 관리하며, 공격 속도 증가, 돌진형 스킬, 멀티 Hit 스킬의 특성에 따른 전투 기능을 타이머와 상태 플래그로 제어해서 구현했습니다.

또한 플레이어의 공격 상태를 정확히 추적하고 판단해서 서버, 클라이언트간 동기화가 원활히 이루어지도록 설계하였습니다.

전체 코드

<https://github.com/sungone/GP2025/blob/main/Source/GP2025/Character/Modules/GPCharacterCombatHandler.h>

```
// 기본 공격
void PlayAutoAttackMontage();
void OnAutoAttackMontageEnded(UAnimMontage* Montage, bool bInterrupted);

// 몬스터 피격
void PlayMonsterHitMontage();

// 사망
void HandleDeath();
void PlayDeadAnimation();
void SetDeadMontage(UAnimMontage* Montage);
void SetDeadEventDelay(float Delay);

// 스킬
void PlayQSkillMontage();
void PlayESkillMontage();
void PlayRSkillMontage();
void OnSkillMontageEnded(UAnimMontage* Montage, bool bInterrupted);

// 애니메이션 세팅
void SetAttackWithoutWeaponMontage(UAnimMontage* Montage);
void SetAttackMontage(UAnimMontage* Montage);
void SetAttackMontageForWarrior(UAnimMontage* Montage1, UAnimMontage* Montage2, UAnimMontage* Montage3);
void SetQSkillMontage(UAnimMontage* Montage);
void SetESkillMontage(UAnimMontage* Montage);
void SetRSkillMontage(UAnimMontage* Montage);
void SetMonsterHitMontage(UAnimMontage* Montage);

// 상태 조회 (애니메이션 상태만)
bool IsAutoAttacking() const { return bIsAutoAttacking; }
bool IsUsingSkill() const { return bIsUsingSkill; }

// <GunenrPlayer R Skill Ability>
void ApplyAttackSpeedBoost(float BoostPlayRate, float Duration);
void ResetAttackSpeed();
float DefaultPlayRate = 2.0f;
FTimerHandle AttackSpeedResetTimer;

// <Warrior R Skill Ability>
private:
    int32 RemainingHits = 0;
    float HitInterval = 0.5f;
    FTimerHandle MultiHitTimerHandle;

public:
    void PlayMultiHitSkill(int32 HitCount, float Interval);
    void ExecuteMultiHit();

// <Warrior E Skill Ability>
private:
    FTimerHandle DashTimerHandle;
    bool bIsDashing = false;
    float DashDistance = 600.f; // 돌진 거리
    float DashDuration = 1.f; // 돌진 시간
    float DashElapsedTime = 0.0f;
    FVector DashStartLocation;
    FVector DashEndLocation;

public:
    void StartDash();
    void FinishDash();
    void UpdateDash();
```

티유월드

인벤토리 및 상점 시스템 구현

아이템 데이터는 GPItemTable 을 활용해 관리하여 손쉽게 아이템의 확장과 수정을 가능하게 했습니다.

ItemSlot 클래스를 인벤토리와 상점 양쪽에서 공통적으로 사용하도록 구조화하여 코드의 일관성과 재사용성을 높였습니다.

또한, 인벤토리와 상점에서 공통적으로 활용할 수 있는 기능들은 별도로 모듈화하여 중복 코드를 최소화하고, 이를 통해 유지보수성과 확장성을 고려한 시스템을 구축했습니다.

아이템 코드

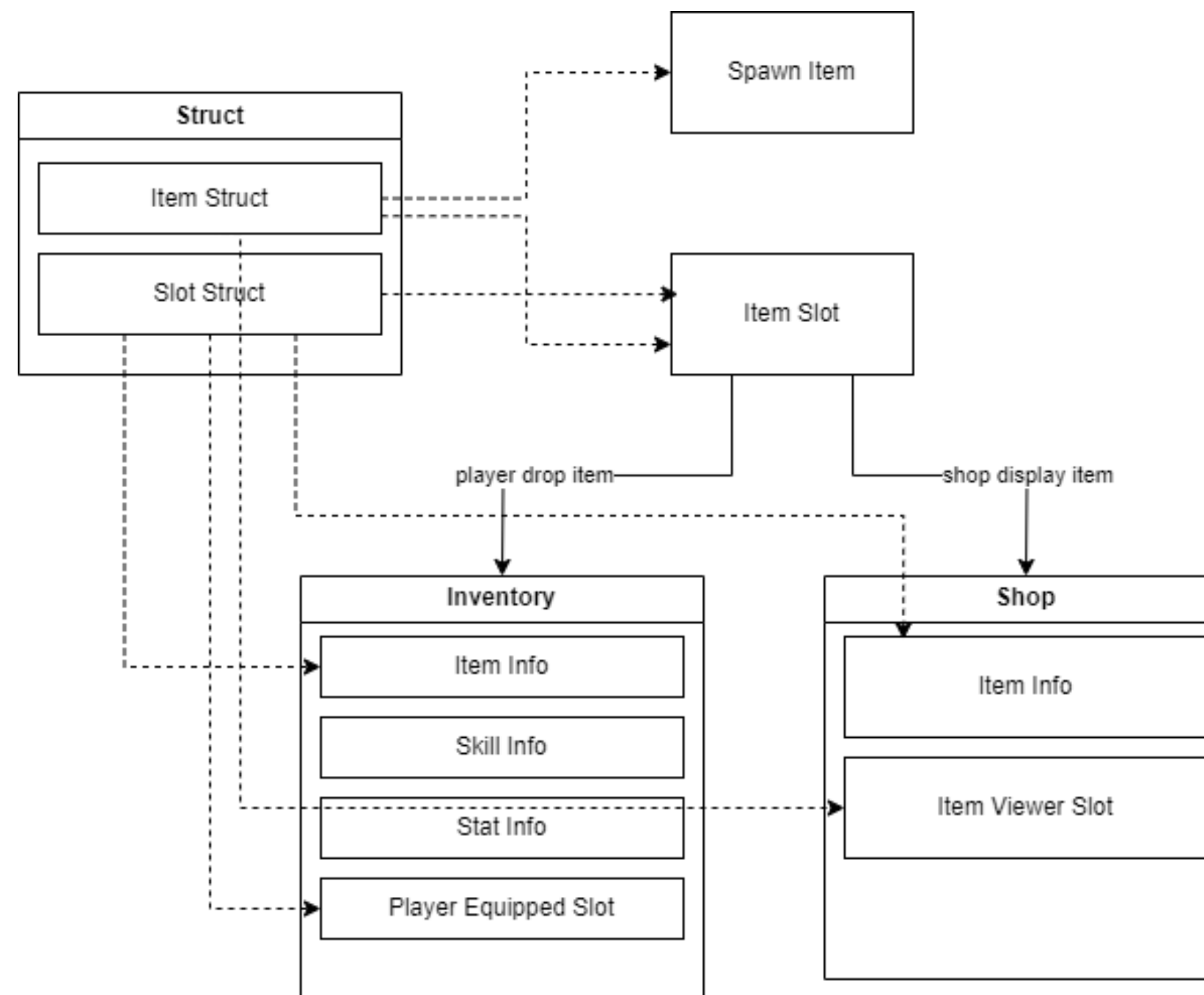
<https://github.com/sungone/GP2025/tree/main/Source/GP2025/Item>

상점 코드

<https://github.com/sungone/GP2025/tree/main/Source/GP2025/Shop>

인벤토리 코드

<https://github.com/sungone/GP2025/tree/main/Source/GP2025/Inventory>



티유월드

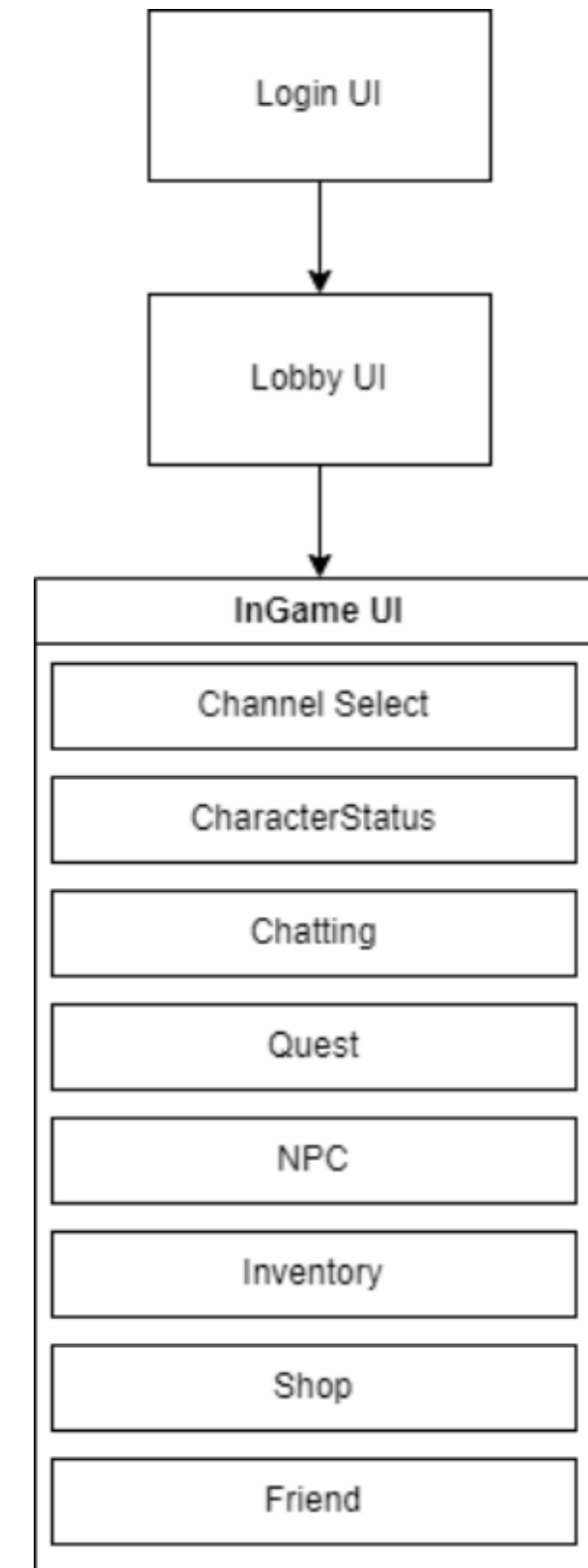
UI 시스템 구현

로그인, 로비, 인게임 UI 를 설계 , 구현하였습니다. 로그인 UI에서 계정 접속을 처리하고, 로비 UI에서 캐릭터 선택과 진입 과정을 거쳐, 인게임 UI에서는 채널 선택, 캐릭터 상태, 채팅, 퀘스트, NPC 상호작용, 인벤토리, 상점, 친구 목록 등 다양한 기능을 모듈화된 위젯 형태로 구현했습니다.

이를 통해 각 UI가 독립적으로 동작하면서도 유기적으로 연결되도록 구조화하여, 확장성과 유지보수성을 고려한 UI 시스템을 구축하였습니다.

전체 코드

<https://github.com/sungone/GP2025/tree/main/Source/GP2025/UI>



Project 소개

Unity Of Mind

2023.09 ~ 2023.12

컴퓨터그래픽스 텀프로젝트

- 벽 피하기 게임
- OpenGL 개발
- 팀 2명 : 클라이언트 2
- 오브젝트 , 셰이더 , 게임 로직 담당
- 다가오는 벽을 피해서 끝까지 생존하는 게임

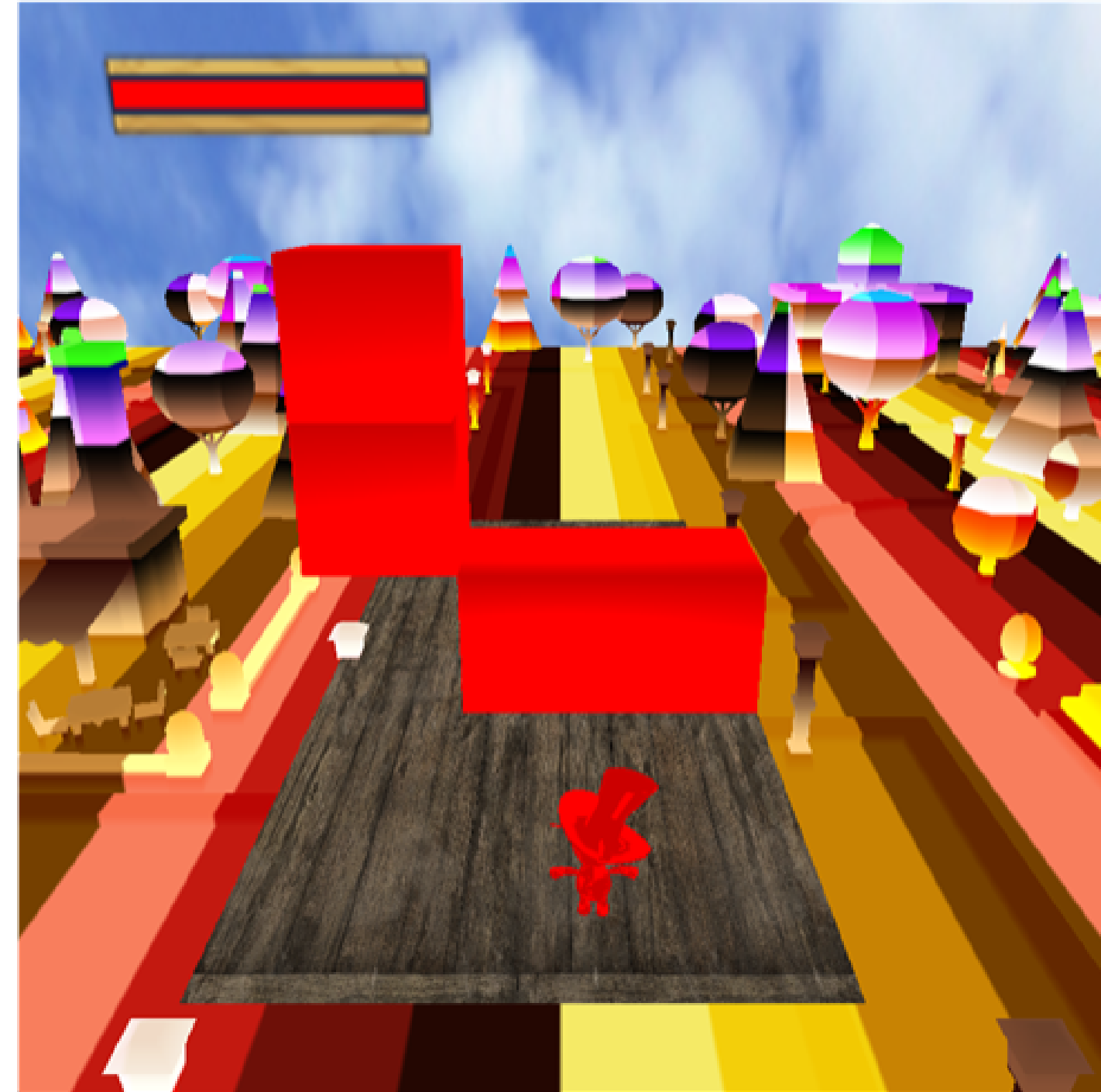
- 깃허브

https://github.com/sungone/Unity_Of_Mind

- 유튜브

[https://www.youtube.com/watch?](https://www.youtube.com/watch?v=9u4resCQxrY)

[v=9u4resCQxrY](https://www.youtube.com/watch?v=9u4resCQxrY)

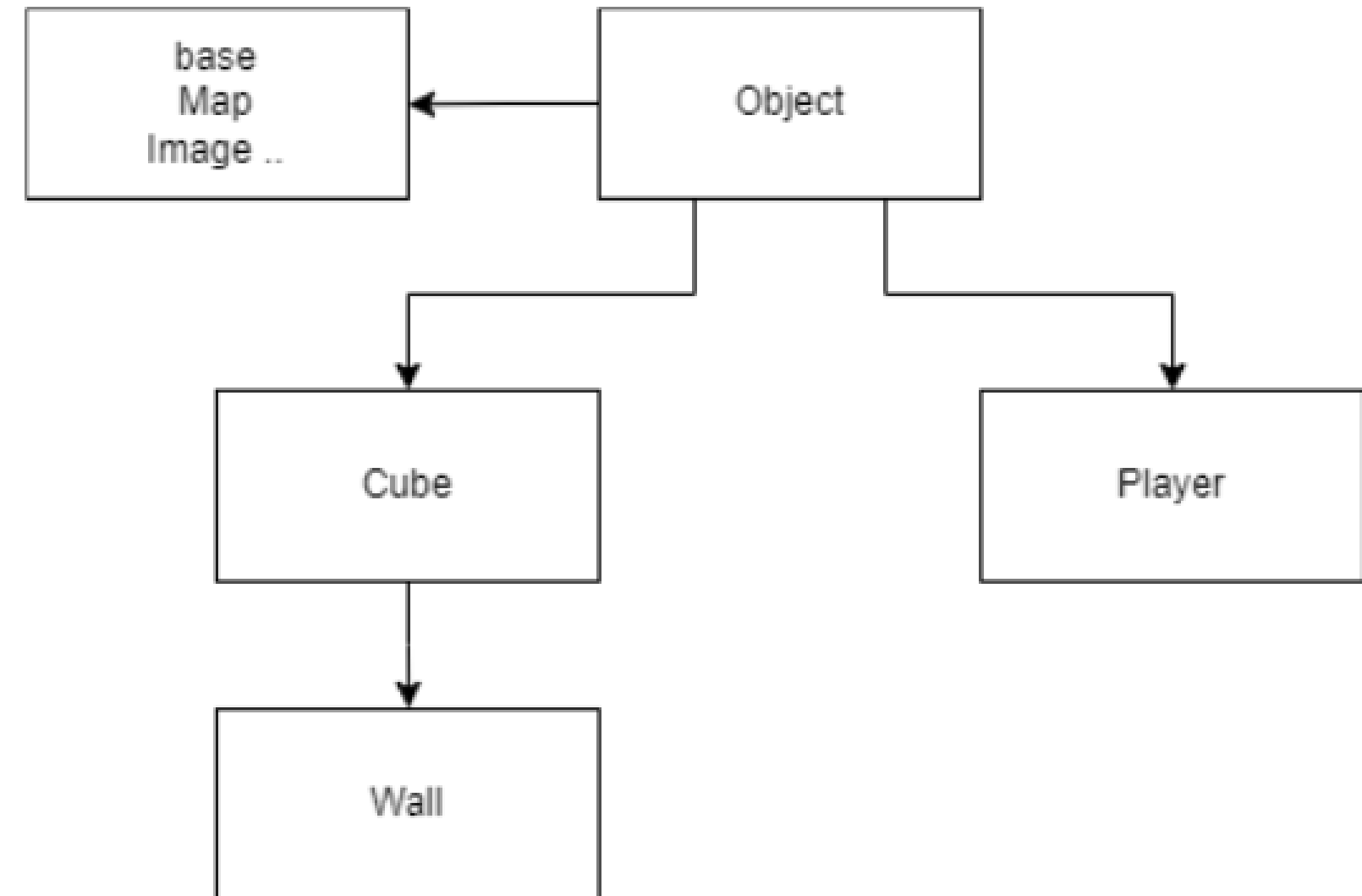


Unity Of Mind

게임 오브젝트 설계

게임의 핵심 오브젝트들을 Object 기반 클래스에서 상속받아 Cube, Wall, Player 등으로 구현하였습니다.

공통 속성과 기능은 Object에서 관리하고, 각 파생 클래스는 개별 동작을 오버라이드하도록 설계하였습니다.



오브젝트 코드

https://github.com/sungone/Unity_Of_Mind/blob/main/CG_FinalProject/object.h

Cube, Wall 코드

https://github.com/sungone/Unity_Of_Mind/blob/main/CG_FinalProject/wall.h

Player 코드

https://github.com/sungone/Unity_Of_Mind/blob/main/CG_FinalProject/player.h

Unity Of Mind

쉐이더 관리 모듈 설계

버텍스 및 프래그먼트 셰이더를 작성하여 오브젝트의 색상과 텍스처를 처리할 수 있도록 했습니다.
이를 통해 Cube, Wall, Player 등 다양한 오브젝트를 렌더링하는 셰이더 시스템을 구축하였습니다.

쉐이더 코드

https://github.com/sungone/Unity_Of_Mind/blob/main/CG_FinalProject/shader.cpp

```
void makeVertexShaders(char* file)
{
    GLchar* vertexSource;

    // char fileName[] = "vertex.glsl";
    // char *file = fileName;
    vertexSource = fileToBuf(file);

    vertexShader = glCreateShader(GL_VERTEX_SHADER);
    glShaderSource(vertexShader, 1, &vertexSource, NULL);
    glCompileShader(vertexShader);

    GLint result;
    GLchar errorLog[512];
    glGetShaderiv(vertexShader, GL_COMPILE_STATUS, &result);
    if (!result)
    {
        glGetShaderInfoLog(vertexShader, 512, NULL, errorLog);
        std::cerr << "Error compiling vertex shader: " << errorLog << std::endl;
        return;
    }
}

void makeFragmentShaders(char* file)
{
    GLchar* fragmentSource;

    // char fileName[] = "fragment.glsl";
    // char *file = fileName;
    fragmentSource = fileToBuf(file);

    fragmentShader = glCreateShader(GL_FRAGMENT_SHADER);
    glShaderSource(fragmentShader, 1, &fragmentSource, NULL);
    glCompileShader(fragmentShader);

    GLint result;
    GLchar errorLog[512];
    glGetShaderiv(fragmentShader, GL_COMPILE_STATUS, &result);
    if (!result)
    {
        glGetShaderInfoLog(fragmentShader, 512, NULL, errorLog);
        std::cerr << "Error compiling fragment shader: " << errorLog << std::endl;
        return;
    }
}
```

감사합니다

조성원

010-9742-4028

sungone000604@gmail.com